

OpenPrescribing in R

Milan Wiedemann

2025-11-28

Table of contents

Overview	3
1 Connect to BigQuery	4
1.1 Get access	4
1.2 Store your credentials	4
1.3 Set up env variable	5
1.4 Connet to BigQuery	5
2 Database schema	7
2.1 Main hscic tables	7
2.1.1 normalised_prescribing	7
2.1.2 regional_teams	7
2.1.3 practices	7
2.1.4 practice_statistics	9
2.2 Relationship diagram?	11
3 Reusable functions	12
3.1 Connect to BigQuery	12
3.2 Data access functions	14
3.2.1 Get practices	14
3.2.2 Get normalised prescribing	17
3.3 Writing SQL with dbplyr	19
4 Applied example	20
References	21

Overview

This tutorial walks you through how to work with the [OpenPrescribing.net](https://openprescribing.net) BigQuery data in R. It will cover:

1. **Connect to BigQuery** from R using [DBI](#)¹ and [bigrquery](#)²
2. **Description of the main tables** in the BigQuery database** [dm](#)³
3. **Reusable functions in R** using [dbplyr](#)⁴
4. **Applied example** following PICK AN EXAMPLE STUDY WE DID

1 Connect to BigQuery

1.1 Get access

To get access to the OpenPrescribing BigQuery database write a message in the #openprescribing Slack channel and tag slack-admin. One of the admins will send you a JSON file with your BigQuery credentials in a private message. The file should look something like this:

```
{
  "type": "ACCOUNT_TYPE",
  "project_id": "PROJECT_ID",
  "private_key_id": "PRIVATE_KEY_ID",
  "private_key": "PRIVATE_KEY",
  "client_email": "CLIENT_EMAIL",
  "client_id": "CLIENT_ID",
  "auth_uri": "AUTH_URI",
  "token_uri": "TOKEN_URI",
  "auth_provider_x509_cert_url": "AUTH_CERTIFICATE",
  "client_x509_cert_url": "CLIENT_CERTIFICATE",
  "universe_domain": "DOMAIN"
}
```

IMPORTANT: Never commit this file to GitHub or share it with anyone.

1.2 Store your credentials

You can store this file anywhere on your computer, but a common location is in a hidden folder in your home directory, for example:

On macOS/Linux:

```
~/.config/openprescribing/bq-credentials.json
```

On Windows:

```
C:/Users/YOUR_USERNAME/.config/openprescribing/bq-credentials.json
```

1.3 Set up env variable

Rather than hardcoding the path in your R scripts you can set an environment variable named `OP_CREDENTIALS` or similar. To help you edit your `.Renviron` file and set the variable for all R sessions you can use the `usethis` package:

```
# Open your .Renviron file
usethis::edit_r_environ()
```

This will open your `.Renviron` file in your editor where you can add the location where you stored your credentials file, for example:

```
# macOS/Linux
OP_CREDENTIALS=~/.config/openprescribing/bq-credentials.json

# Windows
OP_CREDENTIALS=C:/Users/YOUR_USERNAME/.config/openprescribing/bq-credentials.json
```

Note: You will probably need to restart your R session for R to know about your new environment variable.

1.4 Connet to BigQuery

The following code connects to the `hscic` dataset in the `ebmdatalab` project. You will need to authenticate using the path to your credentials file stored in the `OP_CREDENTIALS` environment variable and might be asked to login with your Google account the first time you run this.

```
library(DBI)
library(bigrquery)

bq_auth(path = Sys.getenv("OP_CREDENTIALS"))

con <- dbConnect(
  bigrquery::bigrquery(),
  project = "ebmdatalab",
  dataset = "hscic",
```

```
    billing = "ebmdatalab"  
)
```

2 Database schema

2.1 Main hscic tables

The **hscic** dataset has 194 different tables. Here we describe the main tables used for research projects. To see all the tables available in the **hscic** dataset, you can use:

```
dbListTables(con)
```

2.1.1 normalised_prescribing

Normalised view of the main prescribing dataset, updated monthly. This is the table you'll typically want to query rather than the raw prescribing table.

2.1.2 regional_teams

Open and close dates for regional teams.

2.1.3 practices

Includes all the practices in England, updated monthly. Contains latest month's data only (but includes closed practices). Practices with a **setting** of 4 are standard GP practices. Other settings are:

- 0 = Other
- 1 = WIC Practice
- 2 = OOH Practice
- 3 = WIC + OOH Practice
- 4 = GP Practice
- 8 = Public Health Service
- 9 = Community Health Service
- 10 = Hospital Service
- 11 = Optometry Service
- 12 = Urgent & Emergency Care

Columns		
sha	<i>character</i>	Strategic Health Authority code (e.g., QHM)
regional_team	<i>character</i>	Regional team identifier (e.g., Y63)
stp	<i>character</i>	Sustainability and Transformation Partnership code (e.g., QHM)
pct	<i>character</i>	Primary Care Trust code (e.g., 16C)
practice	<i>character</i>	Practice code (e.g., A81001)
bnf_code	<i>character</i>	British National Formulary (BNF) code (e.g., 0408010W0BBAKA1)
bnf_name	<i>character</i>	British National Formulary (BNF) drug name (e.g., Epilim Chrono 300 tablets)
items	<i>integer</i>	Number of items (one item = one appearance on a prescription)
net_cost	<i>numeric</i>	Cost of the presentation according to the Drug Tariff
actual_cost	<i>numeric</i>	Actual cost including adjustments for bulk purchases, out of pocket expenses, etc. (This is not used in the analysis)
quantity	<i>numeric</i>	Number of pills/grams/millilitres/dressings/ampoules prescribed
month	<i>POSIXct</i>	Prescribing month
Rows 10 Columns 12		

Columns		
code	<i>character</i>	Regional team code (e.g., Y57)
name	<i>character</i>	Regional team name (e.g., SOUTH OF ENGLAND COMMISSIONING REGION)
open_date	<i>Date</i>	Open date
close_date	<i>Date</i>	Closing date, NA if still open
Rows 10 Columns 4		

Columns		
code	<i>character</i>	Practice code (e.g., E82084)
name	<i>character</i>	Practice name (e.g., HATFIELD ROAD SURGERY)
address1	<i>character</i>	Address line 1 (e.g., 13-15 RUSSELL AVENUE)
address2	<i>character</i>	Address line 2 (e.g., LONDON COLNEY)
address3	<i>character</i>	Address line 3 (e.g., LONDON COLNEY)
address4	<i>character</i>	Address line 4 (town/city) (e.g., HATFIELD)
address5	<i>character</i>	Address line 5
postcode	<i>character</i>	Practice postcode (e.g., AL1 3HD)
location	<i>character</i>	Location of practice in WGS84 format (e.g., 0101000020E6100000F24637A67312C)
cgc_id	<i>character</i>	Clinical Commissioning Group identifier (e.g., 06N)
pcn_id	<i>character</i>	Primary Care Network identifier (e.g., U73966)
setting	<i>integer</i>	Practice setting code (4 = standard GP practice)
close_date	<i>character</i>	Date practice closed
join_provider_date	<i>character</i>	Date practice joined CCG
leave_provider_date	<i>character</i>	Date practice left CCG (or closed)
open_date	<i>character</i>	Date practice opened
status_code	<i>character</i>	Practice status code (A=Active, B=Retired, C=Closed, D=Dormant, P=Proposed)
Rows 100 Columns 17		

- 13 = Hospice
- 14 = Care Home / Nursing Home
- 15 = Border Force
- 16 = Young Offender Institution
- 17 = Secure Training Centre
- 18 = Secure Children's Home
- 19 = Immigration Removal Centre
- 20 = Court
- 21 = Police Custody
- 22 = Sexual Assault Referral Centre (SARC)
- 24 = Other

2.1.4 practice_statistics

Total list size and list sizes stratified by gender and age group for each practice, updated monthly. Contains latest month's data only. Also contains calculated population weightings related to prescribing (STAR-PU and ASTRO-PU).

Columns		
month	<i>POSIXct</i>	The month for which practice statistics are recorded
male_0_4	<i>integer</i>	Count of male patients aged 0-4 years
female_0_4	<i>integer</i>	Count of female patients aged 0-4 years
male_5_14	<i>integer</i>	Count of male patients aged 5-14 years
male_15_24	<i>integer</i>	Count of male patients aged 15-24 years
male_25_34	<i>integer</i>	Count of male patients aged 25-34 years
male_35_44	<i>integer</i>	Count of male patients aged 35-44 years
male_45_54	<i>integer</i>	Count of male patients aged 45-54 years
male_55_64	<i>integer</i>	Count of male patients aged 55-64 years
male_65_74	<i>integer</i>	Count of male patients aged 65-74 years
male_75_plus	<i>integer</i>	Count of male patients aged 75 years and over
female_5_14	<i>integer</i>	Count of female patients aged 5-14 years
female_15_24	<i>integer</i>	Count of female patients aged 15-24 years
female_25_34	<i>integer</i>	Count of female patients aged 25-34 years
female_35_44	<i>integer</i>	Count of female patients aged 35-44 years
female_45_54	<i>integer</i>	Count of female patients aged 45-54 years
female_55_64	<i>integer</i>	Count of female patients aged 55-64 years
female_65_74	<i>integer</i>	Count of female patients aged 65-74 years
female_75_plus	<i>integer</i>	Count of female patients aged 75 years and over
total_list_size	<i>integer</i>	Total count of patients registered at the practice
astro_pu_cost	<i>numeric</i>	ASTRO-PU (Age, Sex, and Temporary Resident Originated Prescribing Units) cost wei
astro_pu_items	<i>numeric</i>	ASTRO-PU items weighting
star_pu	<i>character</i>	STAR-PU? as JSON?
pct_id	<i>character</i>	Primary Care Trust identifier (e.g., 08G)
practice	<i>character</i>	Practice code (e.g., E86005)
Rows 10 Columns 25		

2.2 Relationship diagram?

I need someone to check this. Could add some tables from the dmd dataset here (and above)?
Add description here.

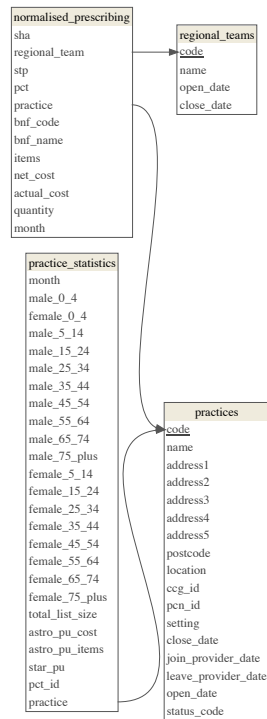


Figure 2.1: Relationship diagram for hscic tables

3 Reusable functions

All functions here are based on the `DBI`, `bigrquery`, `dplyr` and `dbplyr` packages to connect to BigQuery and query the OpenPrescribing data in R.

```
library(DBI)
library(bigrquery)
library(dplyr)
library(dbplyr)
```

3.1 Connect to BigQuery

We could write a simple function to connect to BigQuery that we can reuse in our R scripts. This function assumes that you have set the `OP_CREDENTIALS` environment variable in [Section 1.3](#) and we can define different datasets to connect to by changing the `dataset` argument (default is `hscic`).

```
connect_openprescribing <- function(dataset = "hscic") {
  # Get BigQuery credentials path from environment variable
  credentials_path <- Sys.getenv("OP_CREDENTIALS")

  # Authorise BigQuery
  bigrquery::bq_auth(path = credentials_path)

  # Connect to BigQuery
  DBI::dbConnect(
    bigrquery::bigrquery(),
    project = "ebmdatalab",
    dataset = dataset,
    billing = "ebmdatalab"
  )
}
```

A slightly more robust and flexibly function could also have an option to work with different environment variables and return helpful error messages, see the example below.

```

#' Connect to BigQuery
connect_bq <- function(
  project = "ebmdatalab",
  dataset = "hscic",
  billing = "ebmdatalab",
  credentials_var = "OP_CREDENTIALS"
) {
  # Get the credentials path from environment variable
  credentials_path <- Sys.getenv(credentials_var)

  # Check if environment variable is set
  if (credentials_path == "") {
    cli::cli_abort(c(
      paste("Environment variable", credentials_var, "is not set."),
      "i" = "Please set it in your .Renviron file to point to your BigQuery credentials JSON",
      "i" = "You can edit your .Renviron file using: usethis::edit_r_environ()",
      "i" = "See chapters/connect-bq.qmd for detailed setup instructions."
    ))
  }

  # Check if the credentials file exists
  if (!file.exists(credentials_path)) {
    cli::cli_abort(c(
      paste("Credentials file not found at:", credentials_path),
      "i" = paste(
        "The",
        credentials_var,
        "environment variable is set, but the file does not exist."
      ),
      "i" = "Please check the path in your .Renviron file."
    ))
  }

  # Authenticate with BigQuery
  cli::cli_alert_info("Authenticating with BigQuery ...")
  bigrquery::bq_auth(path = credentials_path)

  # Connect to BigQuery
  cli::cli_alert_info(paste0(
    "Connecting to project ",
    project,
    ", dataset ",

```

```

    dataset
  ))

  con <- DBI::dbConnect(
    bigrquery::bigquery(),
    project = project,
    dataset = dataset,
    billing = billing
  )

  cli::cli_alert_success("Successfully connected to BigQuery!")
  invisible(con)
}

```

However, the simple example would do the job and we can establish a connection like this:

```
con <- connect_openprescribing()
```

3.2 Data access functions

These functions provide a modular way to access and analyse OpenPrescribing data using dbplyr. Each function returns a lazy query that can be composed with other dplyr functions. For a more detailed overview see [Introduction to dbplyr](#).

I'll demonstrate this how this might be useful for two commonly used tables: `practices` and `normalised_prescribing`.

3.2.1 Get practices

For a description of the `practices` table see Section [2.1.3](#).

```

get_practices <- function(
  con,
  add_setting_labels = FALSE,
  filter_setting = NULL,
  add_status_code_labels = FALSE,
  filter_status_code = NULL
) {

  practices_query <- dplyr::tbl(con, "practices")

```

```

# Add status code description labels
if (add_status_code_labels) {
  practices_query <- practices_query |>
    dplyr::mutate(
      status_code_label = dplyr::case_when(
        status_code == "A" ~ "Active",
        status_code == "B" ~ "Retired",
        status_code == "C" ~ "Closed",
        status_code == "D" ~ "Dormant",
        status_code == "P" ~ "Proposed",
        status_code == "U" ~ "Unknown",
        .default = "Unknown"
      )
    )
}

# Add setting description labels
if (add_setting_labels) {
  practices_query <- practices_query |>
    dplyr::mutate(
      setting_label = dplyr::case_when(
        setting == 0 ~ "Other",
        setting == 1 ~ "WIC Practice",
        setting == 2 ~ "OOH Practice",
        setting == 3 ~ "WIC + OOH Practice",
        setting == 4 ~ "GP Practice",
        setting == 8 ~ "Public Health Service",
        setting == 9 ~ "Community Health Service",
        setting == 10 ~ "Hospital Service",
        setting == 11 ~ "Optometry Service",
        setting == 12 ~ "Urgent & Emergency Care",
        setting == 13 ~ "Hospice",
        setting == 14 ~ "Care Home / Nursing Home",
        setting == 15 ~ "Border Force",
        setting == 16 ~ "Young Offender Institution",
        setting == 17 ~ "Secure Training Centre",
        setting == 18 ~ "Secure Childrens Home",
        setting == 19 ~ "Immigration Removal Centre",
        setting == 20 ~ "Court",
        setting == 21 ~ "Police Custody",
        setting == 22 ~ "Sexual Assault Referral Centre (SARC)",
        setting == 24 ~ "Other - Justice Estate",

```

```

        setting == 25 ~ "Prison",
        setting == 26 ~ "PCN",
        setting == 27 ~ "Independent Pharmacy",
        .default = "Unknown"
    )
)
}

# Filter by setting codes
if (!is.null(filter_setting)) {
  practices_query <- practices_query |>
    dplyr::filter(setting %in% filter_setting)
}

# Filter by setting codes
if (!is.null(filter_status_code)) {
  practices_query <- practices_query |>
    dplyr::filter(status_code %in% filter_status_code)
}

practices_query
}

```

Now we can use this function to get practices data using the different options we defined in the function above. Here are the first 5 rows of some columns we can select using the `select()` function from `dplyr`, unfiltered and without labels:

```

get_practices(con) |>
  select(code, name, postcode, setting, status_code) |>
  head(5)

```

```

# Source:   SQL [?? x 5]
# Database: BigQueryConnection
   code    name  postcode setting status_code
  <chr>  <chr>  <chr>      <int>  <chr>
1 01Y999 <NA>  <NA>         -1 <NA>
2 09X999 <NA>  <NA>         -1 <NA>
3 NHT999 <NA>  <NA>         -1 <NA>
4 04R999 <NA>  <NA>         -1 <NA>
5 04E999 <NA>  <NA>         -1 <NA>

```


We can specify to add labels for the setting codes and filter to only include standard GP practices (setting code 4).

```
get_practices(con, add_setting_labels = TRUE, filter_setting = 4) |>
  select(code, name, postcode, setting, setting_label, status_code) |>
  head(5)
```

```
# Source:   SQL [?? x 6]
# Database: BigQueryConnection
   code    name                postcode setting setting_label status_code
   <chr>   <chr>                <chr>    <int> <chr>         <chr>
1 E82004  HATFIELD ROAD SURGERY    AL1 3FY      4 GP Practice  A
2 E82060  PARKBURY HOUSE SURGERY    AL1 3HD      4 GP Practice  A
3 E82031  MALTINGS SURGERY          AL1 3JB      4 GP Practice  A
4 E82002  WRAFTON HOUSE SURGERY     AL10 OBS     4 GP Practice  A
5 E82018  LISTER HOUSE SURGERY      AL10 ONL     4 GP Practice  A
```

```
get_practices(con, add_setting_labels = TRUE, filter_setting = 4,
              add_status_code_labels = TRUE) |>
  select(code, name, postcode, setting, setting_label, status_code, status_code_label) |>
  head(5)
```

```
# Source:   SQL [?? x 7]
# Database: BigQueryConnection
   code    name                postcode setting setting_label status_code status_code_label
   <chr>   <chr>                <chr>    <int> <chr>         <chr>         <chr>
1 E82004  HATFIELD ~ AL1 3FY      4 GP Practice  A             Active
2 E82060  PARKBURY ~ AL1 3HD      4 GP Practice  A             Active
3 E82031  MALTINGS ~ AL1 3JB      4 GP Practice  A             Active
4 E82002  WRAFTON H~ AL10 OBS     4 GP Practice  A             Active
5 E82018  LISTER HO~ AL10 ONL     4 GP Practice  A             Active
```

3.2.2 Get normalised prescribing

For a description of the `normalised_prescribing` table see Section 2.1.1. This function returns normalised prescribing data with optional filtering by BNF codes and date range.

```
get_normalised_prescribing <- function(
  con,
  bnf_codes = NULL,
```

```

start_date = NULL,
end_date = NULL) {

  np_query <- dplyr::tbl(con, "normalised_prescribing")

  # Filter by BNF codes if provided
  if (!is.null(bnf_codes)) {
    np_query <- np_query |>
      dplyr::filter(bnf_code %in% !!bnf_codes)
  }

  # Filter by start date if provided
  if (!is.null(start_date)) {
    np_query <- np_query |>
      dplyr::filter(month >= !!start_date)
  }

  # Filter by end date if provided
  if (!is.null(end_date)) {
    np_query <- np_query |>
      dplyr::filter(month <= !!end_date)
  }

  np_query
}

```

In this example we get normalised prescribing data for two BNF codes between January and March 2023.

```

get_normalised_prescribing(
  con,
  bnf_codes = c("0408010WOBBAKA1", "0601022B0AAASAS"),
  start_date = "2023-01-01",
  end_date = "2023-03-01"
) |>
  filter(practice == "D83033") |>
  head(5)

```

```

# Source:   SQL [?? x 12]
# Database: BigQueryConnection
  sha      regional_team stp    pct    practice bnf_code      bnf_name items net_cost
<chr> <chr>          <chr> <chr> <chr>      <chr>      <chr>    <int>    <dbl>

```

1	QJG	Y61	QJG	06L	D83033	0408010WOBBA~	Epilim ~	5	68.1
2	QJG	Y61	QJG	06L	D83033	0408010WOBBA~	Epilim ~	6	68.1
3	QJG	Y61	QJG	06L	D83033	0601022B0AAA~	Metform~	59	105.
4	QJG	Y61	QJG	06L	D83033	0601022B0AAA~	Metform~	62	104.
5	QJG	Y61	QJG	06L	D83033	0408010WOBBA~	Epilim ~	6	78.6

i 3 more variables: actual_cost <dbl>, quantity <dbl>, month <dtm>

3.3 Writing SQL with dbplyr

See [Writing SQL with dbplyr](#).

4 Applied example

Replicate some work here that illustrates how nice it is to do this in R.

References

1. R Special Interest Group on Databases (R-SIG-DB), Wickham, H. & Müller, K. *DBI: R Database Interface*. (2024). doi:[10.32614/CRAN.package.DBI](https://doi.org/10.32614/CRAN.package.DBI).
2. Wickham, H. & Bryan, J. *bigrquery: An Interface to Google's 'BigQuery' 'API'*. (2025). doi:[10.32614/CRAN.package.bigrquery](https://doi.org/10.32614/CRAN.package.bigrquery).
3. Schieferdecker, T., Müller, K. & Bergant, D. *dm: Relational Data Models*. (2025). doi:[10.32614/CRAN.package.dm](https://doi.org/10.32614/CRAN.package.dm).
4. Wickham, H., Girlich, M. & Ruiz, E. *dbplyr: A 'Dplyr' Back End for Databases*. (2024). doi:[10.32614/CRAN.package.dbplyr](https://doi.org/10.32614/CRAN.package.dbplyr).